📘 Polling vs Webhooks vs WebSockets

Chapter 3 — WebSockets (Zero → Hero)

1. The Problem

Let's build WhatsApp.

Suppose User A sends

```
Hello
```

Question:

How will User B receive it?

OPTION 1

Polling.

User B asks every

```
5 seconds
```

```
Phone

↓

GET /messages

↓

No Messages
```

Again

```
GET /messages

↓
```

```
No Messages
```

Again

```
GET /messages

↓

Hello
```

Problem

Suppose User A sends

```
Hello
```

at

```
10:00:01
```

User B polls

```
10:00:05
```

Message arrives

```
4 seconds late
```

Now imagine

Typing...

Blue Tick

Online

Last Seen

Voice Call

Video Call

Impossible with polling.

---

2. Why Not Webhooks?

Some people say

> Use Webhooks.

Wrong.

Remember

Webhook works

```
Server

↓

Server
```

Browser

or

Mobile App

cannot expose

```
https://phone123.com/webhook
```

because

- phones keep changing networks
- browsers are behind NAT
- devices don't expose public endpoints

Therefore

Webhooks are not for mobile apps.

They are for

```
Stripe

↓

Your Backend
```

---

3. The Big Idea

Instead of creating

new HTTP connection

for every message

keep

ONE

connection alive.

Like a phone call.

Instead of

```
Hello

Call End

Call Again

Hello

Call End
```

You simply stay connected.

---

4. What is a WebSocket?

Definition

A WebSocket is a protocol that creates a **persistent, full-duplex communication channel** over a single TCP connection.

Break this sentence.

Persistent

↓

Connection stays alive.

Full Duplex

↓

Both sides can send data anytime.

Single TCP Connection

↓

No new connection every request.

Visualization

```
Client

====================

Server
```

Connection never closes.

---

5. How Does It Start?

Interview Favorite.

WebSocket doesn't magically appear.

It starts as

HTTP.

Browser sends

```
GET /chat HTTP/1.1

Upgrade: websocket

Connection: Upgrade
```

Browser says

> Can we upgrade?

Server replies

```
HTTP/1.1 101 Switching Protocols
```

Very famous status code.

After this

HTTP ends.

WebSocket begins.

Visualization

```
HTTP Request

↓

Upgrade Request

↓

101 Switching Protocols

↓

WebSocket Connection
```

Interview Trap

Question

Is WebSocket built on HTTP?

Correct Answer

No.

It **starts using HTTP for the handshake**, then switches to the **WebSocket protocol** over the same TCP connection.

---

6. What Happens Next?

Now

both sides can send messages.

No request.

No response.

Just messages.

```
Client

<==================>

Server
```

Client

↓

Hello

Server

↓

Hi

Client

↓

Typing...

Server

↓

Delivered

Server

↓

Online

Client

↓

Seen

Everything over

same connection.

---

7. Full Duplex

Another interview favorite.

HTTP

```
Client

↓

Request

↓
```

```
Server

↓

Response
```

Server cannot suddenly send

"Hello"

without request.

WebSocket

```
Client

<================>

Server
```

Both sides speak whenever they want.

Exactly like a phone call.

---

8. WebSocket Frames

Many people think

WebSocket sends HTTP requests.

Wrong.

After handshake

WebSocket sends

Frames.

Example

```
Frame

↓

Opcode
```

```
↓

Payload

↓

Mask

↓

Length
```

Payload

may contain

```
Hello
```

or

```
{
 "typing":true
}
```

These are lightweight compared to HTTP requests.

---

9. Why Is It Faster?

HTTP Request

contains

```
Headers

Cookies

Authorization

User Agent

Accept

Host
```

```
...
```

Every request repeats them.

WebSocket

Handshake only once.

Then

only message.

```
Hello

Typing

Seen

Online
```

Less bandwidth.

Less latency.

---

10. Ping/Pong

Suppose

User closed laptop.

Internet disconnected.

Server doesn't know.

Connection still exists.

How to detect?

Use Heartbeats.

Server sends

```
PING
```

Client replies

```
PONG
```

No Pong?

Connection dead.

Close it.

Visualization

```
Server
PING
↓
Client
PONG
↓
Server
PING
↓
```

Interview Question

Why Ping/Pong?

Expected Answer

To detect dead connections and keep idle connections alive through intermediaries.

---

11. Reconnection

Suppose

WiFi disconnects.

Connection breaks.

Client automatically

tries again.

```
Connect
↓
Disconnected
↓
Retry
↓
Retry
↓
Connected
```

WhatsApp does this.

Slack does this.

Discord does this.

---

12. Scaling Problem

One user

One WebSocket.

Easy.

Now imagine

10 Million Users.

Each has

one connection.

Server stores

10 million open sockets.

Huge memory.

Huge file descriptors.

Huge networking.

Therefore

companies use

multiple WebSocket servers.

```
            Load Balancer

           /      |      \

      WS1     WS2     WS3
```

Each server handles

only part of users.

---

13. Sticky Sessions

Question

User connected to

WS Server 1.

Next packet

goes to

WS Server 3.

Problem.

WS3 doesn't know

that connection.

Therefore

Load Balancer usually keeps

same client

on same WebSocket server.

This is called

Sticky Session.

---

14. Production Architecture

WhatsApp doesn't look like

```
Phone

↓

Server
```

It looks like

```
User A

↓

WebSocket Gateway

↓

Kafka

↓

Message Service

↓

WebSocket Gateway
```

Notice

WebSocket

is only the communication layer.

Business logic

still happens

through

Kafka,

Redis,

Databases,

Caches.

Huge interview point.

---

15. Real Company Examples

WHATSAPP

Messages

Typing

Blue Tick

SLACK

Realtime Chat

DISCORD

Voice

Chat

Presence

TRADING APPS

Live Prices

GOOGLE DOCS

Live Editing

FIGMA

Realtime Collaboration

---

16. When NOT to Use WebSockets

Don't use WebSockets

for

✔ Payment Success

✔ Order Confirmation Email

✔ GitHub Push

✔ Background Jobs

Those are

event-driven

not continuous communication.

Webhook is better.

---

17. Polling vs Webhook vs WebSocket

Feature	Polling	Webhook	WebSocket
Who starts communication?	Client	Server	Both
Protocol	HTTP	HTTP	WebSocket
Connection	New every request	New per event	Persistent
Communication	One-way	One-way	Two-way
Real-time	❌ Limited	✅ Event-driven	✅ Best
Latency	Medium–High	Low	Very Low
Server Load	High	Low	Medium (many open connections)
Best For	Dashboards, Job Status	Payments, Git Push	Chat, Live Updates

---

18. Which One Should I Choose?

POLLING

Choose when

Updates are rare.

Simple solution needed.

Example

Report Generation.

WEBHOOK

Choose when

One backend

must notify

another backend.

Example

Stripe

GitHub

Shopify

WEBSOCKET

Choose when

Communication is continuous.

Realtime.

Bidirectional.

Example

WhatsApp

Slack

Trading

Gaming

---

19. MAANG Interview Questions

Q1

Why not use Polling instead of WebSocket?

Expected

Too many unnecessary HTTP requests.

Higher latency.

Poor scalability.

Q2

Why not use Webhooks?

Expected

Mobile apps

cannot expose

public HTTP endpoints.

Q3

Why is WebSocket faster?

Expected

Single TCP connection.

No repeated HTTP handshake.

Small frames.

Lower latency.

Q4

Why Ping/Pong?

Expected

Detect dead connections.

Keep connection alive.

Q5

Can WebSocket replace REST APIs?

Expected

No.

REST is still better

for CRUD operations.

WebSocket is for realtime communication.

Q6

Does WebSocket use TCP?

Yes.

WebSocket runs over TCP.

This guarantees

ordered,

reliable

delivery.

---

20. Common Misconceptions

❌ "WebSocket is always better."

No.

If updates happen once every hour,

Polling is simpler.

❌ "Webhook is for frontend."

No.

Webhook is usually

Server → Server.

❌ "WebSocket replaces REST."

No.

Most production systems use both.

REST

-

WebSocket.

---

21. Senior Engineer Insight (4–10 YOE)

A junior engineer thinks:

> "We need realtime. Let's use WebSockets."

A senior engineer asks:

- How many concurrent connections?
- How will you reconnect?
- How will you authenticate?
- How will you scale gateways?
- What happens if one gateway crashes?
- Do we need sticky sessions?
- How do we broadcast to millions of users?
- How will Kafka/Redis integrate with WebSocket servers?

The protocol is only **10%** of the problem. Operating it reliably at scale is the real engineering challenge.

Polling: Polling is a client-driven communication mechanism where the client repeatedly sends HTTP requests to check for updates. It is simple to implement but generates unnecessary requests and higher server load. It is commonly used for dashboards, report generation, and job status tracking.

Webhook: A Webhook is an event-driven HTTP callback where one server automatically notifies another server when an event occurs. It is efficient because requests are sent only when needed, but it requires retries, idempotency, and security checks. It is widely used for payments, GitHub events, Shopify, and Twilio.

WebSocket: WebSocket is a protocol that creates a persistent TCP connection, allowing both the client and server to exchange data in real time. It provides low latency and is ideal for chat applications, live notifications, collaborative editing, trading platforms, and online gaming.